

STATS_506_PS2

Zixun Zhang

Github repository link: <https://github.com/zixunzhang99/stats506>

Problem 1: Random Walk with Momentum

a. Simulation function:

```
#' Simulate a two-dimensional random walk with momentum
#'
```

```
#' Simulates a random walk starting at (0, 0). The first direction is
#'
```

```
#' chosen randomly. For each subsequent step, the previous direction is
#'
```

```
#' preserved with probability p; otherwise, one of the four directions
#'
```

```
#' is chosen randomly.
#'
```

```
#' @param n A positive integer representing the number of steps.
#'
```

```
#' @param p A numeric value between 0 and 1 representing the probability
#'
```

```
#' of preserving the previous direction.
#'
```

```
#' @return A numeric vector of length two containing the final x and y
#'
```

```
#' coordinates of the particle.
momentum_walk <- function(n, p) {
  if (!is.numeric(n) || length(n) != 1 || n < 1 || n %% 1 != 0) {
    stop("n must be a positive integer")
  }

  if (!is.numeric(p) || length(p) != 1 || p < 0 || p > 1) {
    stop("p must be a number between 0 and 1")
  }

  directions <- matrix(
    c(1, 0,
      -1, 0,
      0, 1,
      0, -1),
    nrow = 4,
    byrow = TRUE
  )
```

```

position <- c(0, 0)

current_dir <- sample(1:4, 1)
position <- position + directions[current_dir, ]

if (n > 1) {
  for (i in 2:n) {

    if (runif(1) > p) {
      current_dir <- sample(1:4, 1)
    }

    position <- position + directions[current_dir, ]
  }
}

return(position)
}

```

```
momentum_walk(10, 0.6)
```

```
[1] -4  2
```

```
momentum_walk(10, 0.6)
```

```
[1]  1 -3
```

b. Implement the simulation

i. Version 1: write the function `momentum_walk1(n, p)` using a loop (a `for` loop or a `while` loop).

```

#' Simulate a two-dimensional random walk with momentum
#'
#' Simulates a random walk starting at (0, 0). The first direction is
#' chosen randomly. For each subsequent step, the previous direction is
#' preserved with probability p; otherwise, one of the four directions
#' is chosen randomly.
#'
#' @param n A positive integer representing the number of steps.
#' @param p A numeric value between 0 and 1 representing the probability

```

```

#'   of preserving the previous direction.
#'
#' @return A numeric vector of length two containing the final x and y
#'   coordinates of the particle.
momentum_walk1 <- function(n, p) {
  if (!is.numeric(n) || length(n) != 1 || n < 1 || n %% 1 != 0) {
    stop("n must be a positive integer")
  }

  if (!is.numeric(p) || length(p) != 1 || p < 0 || p > 1) {
    stop("p must be a number between 0 and 1")
  }

  directions <- matrix(
    c(1, 0,
      -1, 0,
      0, 1,
      0, -1),
    nrow = 4,
    byrow = TRUE
  )

  position <- c(0, 0)

  current_dir <- sample(1:4, 1)
  position <- position + directions[current_dir, ]

  if (n > 1) {
    for (i in 2:n) {

      if (runif(1) > p) {
        current_dir <- sample(1:4, 1)
      }

      position <- position + directions[current_dir, ]
    }
  }

  return(position)
}

```

For `momentum_walk1()`, a `for` loop is used to simulate the walk one step at a time. The first direction is selected randomly. For each subsequent step, the function either preserves the previous

direction with probability (p), or randomly selects one of the four directions with probability (1-p). The particle's position is updated after each step.

ii. **Version 2: write the function `momentum_walk2(n, p)` using one of the apply functions (choose an appropriate member of the apply family).**

```
#' Simulate a random walk with momentum using an apply function
#'
#' Simulates a two-dimensional random walk starting at the origin.
#' The first direction is chosen randomly. For each subsequent step,
#' the particle preserves its previous direction with probability `p`;
#' otherwise, a new direction is chosen randomly from the four possible
#' directions.
#'
#' @param n A positive integer specifying the number of steps.
#' @param p A numeric value between 0 and 1 specifying the probability
#'   of preserving the previous direction.
#'
#' @return A numeric vector of length 2 containing the final x and y
#'   coordinates of the particle.
momentum_walk2 <- function(n, p) {
  if (!is.numeric(n) || length(n) != 1 || n < 1 || n %% 1 != 0) {
    stop("n must be a positive integer")
  }

  if (!is.numeric(p) || length(p) != 1 || p < 0 || p > 1) {
    stop("p must be a number between 0 and 1")
  }

  directions <- matrix(
    c(1, 0,
      -1, 0,
      0, 1,
      0, -1),
    nrow = 4,
    byrow = TRUE
  )

  current_dir <- sample(1:4, 1)

  position <- directions[current_dir, ]

  if (n == 1) {
    return(position)
  }
}
```

```

}

steps <- sapply(2:n, function(i) {

  if (runif(1) >= p) {
    current_dir <- sample(1:4, 1)
  }

  directions[current_dir, i]
})

position <- position + rowSums(steps)

return(position)
}

```

For `momentum_walk2()`, `sapply()` is used to generate the subsequent steps instead of an explicit loop. The function similarly keeps track of the previous direction and either preserves it or randomly selects a new direction at each step. The resulting movements are then summed to obtain the final position.

```
momentum_walk1(10, 0.6)
```

```
[1] -3 -1
```

```
momentum_walk2(10, 0.6)
```

```
[1] -2  2
```

```
momentum_walk1(1000, 0.6)
```

```
[1] -54  32
```

```
momentum_walk2(1000, 0.6)
```

```
[1]  -5 -117
```

c. Reproducibility

```
set.seed(506)
```

```
momentum_walk1(10, 0.6)
```

```
[1] -2  0
```

```
momentum_walk1(1000, 0.6)
```

```
[1] -16 -32
```

I used `set.seed()` to set the initial state of R's random-number generator. Using the same seed before running the simulation causes R to generate the same sequence of random numbers, so the simulation produces the same result and is reproducible.

d. Does momentum matter?

i. For each value of p , simulate a walk of 1000 steps.

```
set.seed(506)
```

```
walk_0 <- momentum_walk1(1000, 0)
```

```
walk_05 <- momentum_walk1(1000, 0.5)
```

```
walk_09 <- momentum_walk1(1000, 0.9)
```

```
dist_0 <- sqrt(sum(walk_0^2))
```

```
dist_05 <- sqrt(sum(walk_05^2))
```

```
dist_09 <- sqrt(sum(walk_09^2))
```

```
data.frame(  
  p = c(0, 0.5, 0.9),  
  distance = c(dist_0, dist_05, dist_09)  
)
```

```
   p distance  
1 0.0 23.19483  
2 0.5 62.28965  
3 0.9 157.00318
```

ii. Does increasing momentum appear to increase or decrease the final distance?

Increasing momentum appears to increase the final distance.

iii. Explain your reasoning

With larger momentum, the particle is more likely to preserve its previous direction, resulting in longer runs in the same direction.

e. Monte Carlo simulation

i. - iii.

```
set.seed(506)

p_values <- c(0, 0.25, 0.5, 0.75, 0.9)

mean_distances <- sapply(p_values, function(p) {

  distances <- replicate(1000, {

    final_pos <- momentum_walk1(1000, p)

    sqrt(sum(final_pos^2))
  })

  mean(distances)
})

results <- data.frame(
  p = p_values,
  mean_distance = mean_distances
)

results
```

	p	mean_distance
1	0.00	27.75439
2	0.25	36.24280
3	0.50	48.41756
4	0.75	75.69467
5	0.90	122.89538

iv. Answer: How does the mean final distance change as p increases?

The mean final distance increases as p increases.

f. Compare computational performance

```
library(microbenchmark)
```

```
benchmark_1000 <- microbenchmark(
  loop = momentum_walk1(1000, 0.6),
  apply = momentum_walk2(1000, 0.6)
)
```

Warning in microbenchmark(loop = momentum_walk1(1000, 0.6), apply = momentum_walk2(1000, : less accurate nanosecond times to avoid potential integer overflows

```
benchmark_1000
```

```
Unit: microseconds
  expr      min       lq      mean   median      uq      max  neval  cld
  loop  958.170 1031.355 1107.198 1061.347 1101.814 1765.501   100    a
  apply 1210.443 1280.512 1348.823 1306.977 1352.898 2206.251   100    b
```

```
benchmark_100000 <- microbenchmark(
  loop = momentum_walk1(100000, 0.6),
  apply = momentum_walk2(100000, 0.6)
)
```

```
benchmark_100000
```

```
Unit: milliseconds
  expr      min       lq      mean   median      uq      max  neval  cld
  loop 103.6949 108.1825 112.4989 110.4381 113.2441 163.8150   100    a
  apply 131.3055 135.2602 139.8630 137.3034 139.5085 182.3151   100    b
```

i. - iii.

The `for` loop implementation is faster. Yes, the difference becomes larger for the larger input. This may occur because the `apply` implementation introduces additional function-call and result-handling overhead. Since each step of the random walk depends on the previous step, the `apply` implementation does not provide a substantial vectorization advantage over the loop.

Problem 2: Monte Carlo Simulation of Campus Bus Arrivals

```
set.seed(506)

n_days <- 10000
```

```

morning <- matrix(
  rpois(n_days * 3, lambda = 4),
  nrow = n_days
)

peak_morning <- rnorm(
  n_days,
  mean = 45,
  sd = sqrt(16)
)

midday <- matrix(
  rpois(n_days * 6, lambda = 15),
  nrow = n_days
)

peak_afternoon <- rnorm(
  n_days,
  mean = 55,
  sd = sqrt(25)
)

evening <- matrix(
  rpois(n_days * 6, lambda = 9),
  nrow = n_days
)

```

```

daily_total <- rowSums(morning) +
  peak_morning +
  rowSums(midday) +
  peak_afternoon +
  rowSums(evening)

mean(daily_total)

```

```
[1] 256.1777
```

The expected total number of passengers who board the bus per day is about 256.

Problem 3: Linear Regression

```
youtube <- read.csv(  
  "https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/  
  2021/2021-03-02/youtube.csv"  
)
```

a. Data preparation and de-identification

```
remove_cols <- c(  
  "brand",  
  "superbowl_ads_dot_com_url",  
  "youtube_url",  
  "id",  
  "kind",  
  "etag",  
  "published_at",  
  "title",  
  "description",  
  "thumbnail",  
  "channel_title"  
)  
  
youtube_clean <- youtube[, !(names(youtube) %in% remove_cols)]  
  
dim(youtube_clean)
```

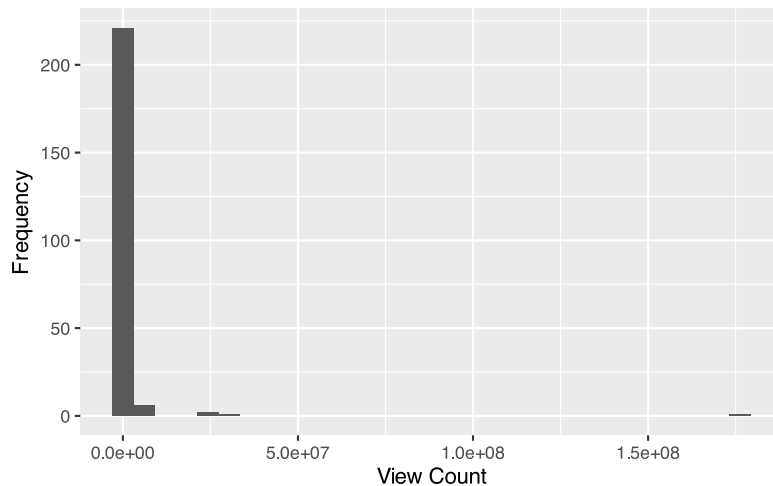
```
[1] 247 14
```

b. Exploring potential outcomes

```
library(ggplot2)  
  
ggplot(youtube_clean, aes(x = view_count)) +  
  geom_histogram() +  
  labs(  
    x = "View Count",  
    y = "Frequency"  
  )
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

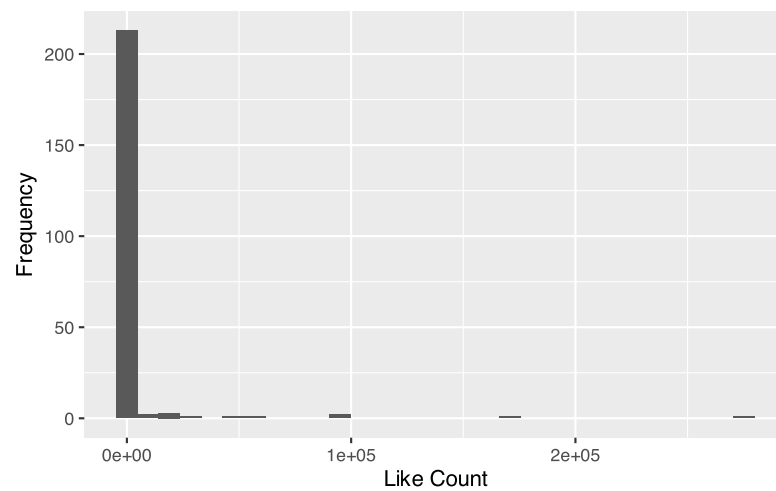
Warning: Removed 16 rows containing non-finite outside the scale range
(`stat\_bin()`).



```
ggplot(youtube_clean, aes(x = like_count)) +  
  geom_histogram() +  
  labs(  
    x = "Like Count",  
    y = "Frequency"  
  )
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

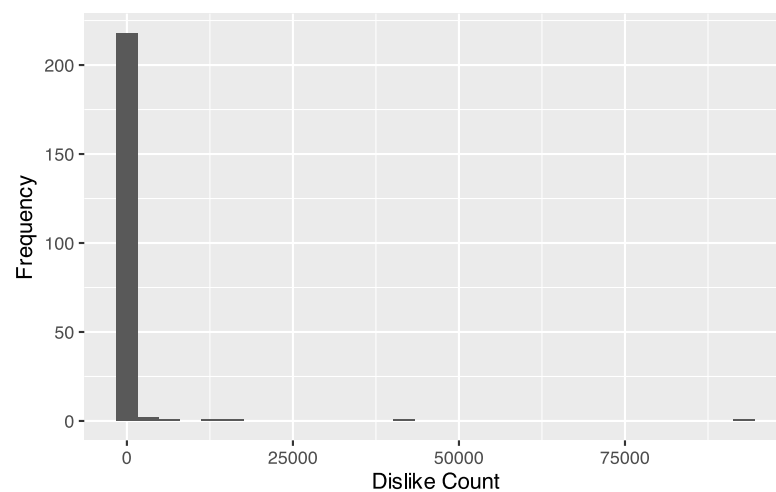
Warning: Removed 22 rows containing non-finite outside the scale range
(`stat\_bin()`).



```
ggplot(youtube_clean, aes(x = dislike_count)) +
  geom_histogram() +
  labs(
    x = "Dislike Count",
    y = "Frequency"
  )
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

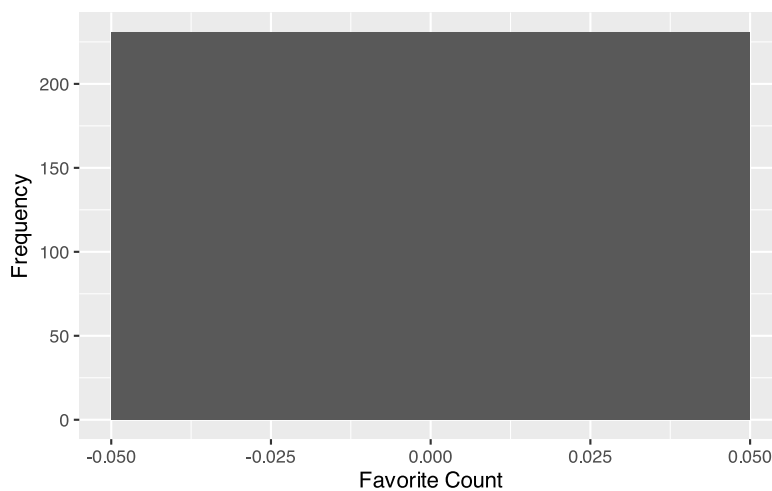
Warning: Removed 22 rows containing non-finite outside the scale range (`stat\_bin()`).



```
ggplot(youtube_clean, aes(x = favorite_count)) +
  geom_histogram() +
  labs(
    x = "Favorite Count",
    y = "Frequency"
  )
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

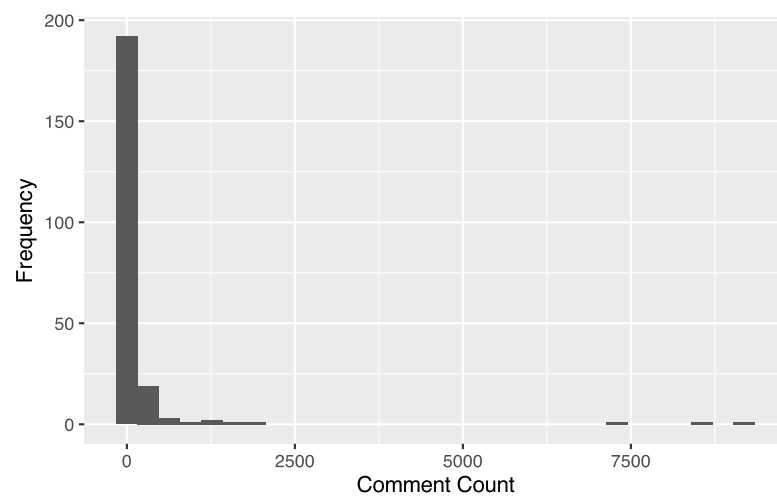
Warning: Removed 16 rows containing non-finite outside the scale range (`stat\_bin()`).



```
ggplot(youtube_clean, aes(x = comment_count)) +
  geom_histogram() +
  labs(
    x = "Comment Count",
    y = "Frequency"
  )
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

Warning: Removed 25 rows containing non-finite outside the scale range (`stat\_bin()`).



```
summary(youtube_clean$view_count)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
10	6431	41379	1407556	170016	176373378	16

```
summary(youtube_clean$like_count)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
0	19	130	4146	527	275362	22

```
summary(youtube_clean$dislike_count)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
0.0	1.0	7.0	833.5	24.0	92990.0	22

```
summary(youtube_clean$favorite_count)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
0	0	0	0	0	0	16

```
summary(youtube_clean$comment_count)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
0.00	1.00	10.00	188.64	50.75	9190.00	25

```
youtube_clean$log_view_count <- log(youtube_clean$view_count)

youtube_clean$sqrt_like_count <- sqrt(youtube_clean$like_count)

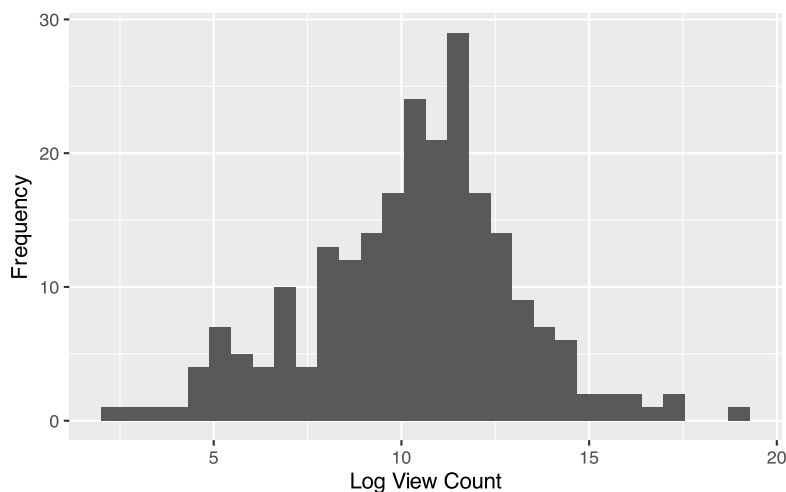
youtube_clean$sqrt_dislike_count <- sqrt(youtube_clean$dislike_count)

youtube_clean$sqrt_comment_count <- sqrt(youtube_clean$comment_count)
```

```
ggplot(youtube_clean, aes(x = log_view_count)) +
  geom_histogram() +
  labs(x = "Log View Count", y = "Frequency")
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

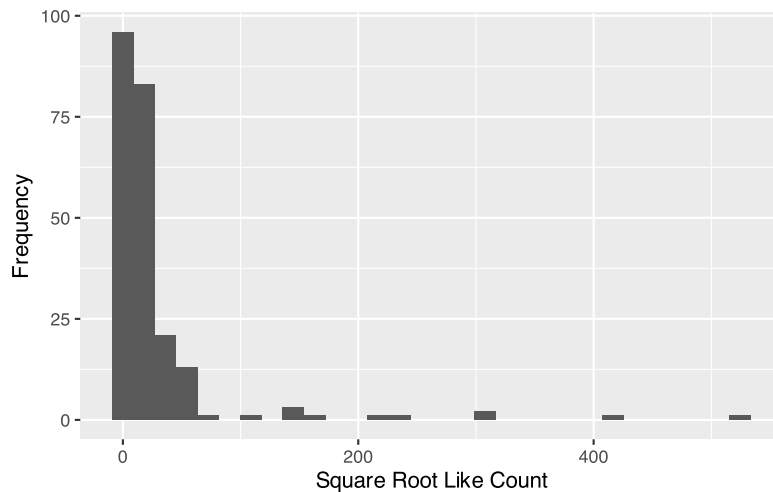
Warning: Removed 16 rows containing non-finite outside the scale range (`stat\_bin()`).



```
ggplot(youtube_clean, aes(x = sqrt_like_count)) +
  geom_histogram() +
  labs(x = "Square Root Like Count", y = "Frequency")
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

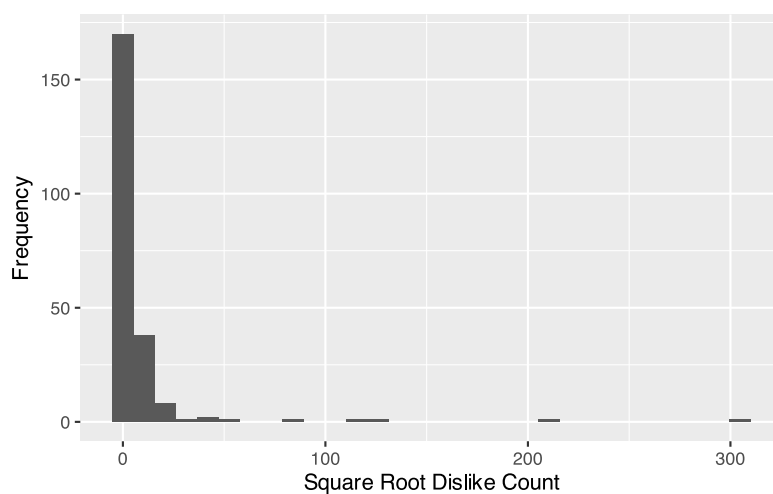
Warning: Removed 22 rows containing non-finite outside the scale range (`stat\_bin()`).



```
ggplot(youtube_clean, aes(x = sqrt_dislike_count)) +  
  geom_histogram() +  
  labs(x = "Square Root Dislike Count", y = "Frequency")
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

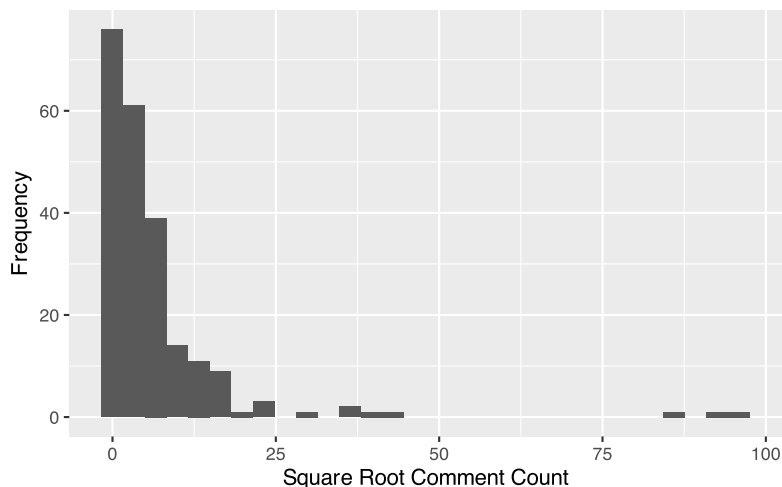
Warning: Removed 22 rows containing non-finite outside the scale range (`stat\_bin()`).



```
ggplot(youtube_clean, aes(x = sqrt_comment_count)) +
  geom_histogram() +
  labs(x = "Square Root Comment Count", y = "Frequency")
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

```
Warning: Removed 25 rows containing non-finite outside the scale range
(`stat_bin()`).
```



The distribution of `view_count` is extremely right-skewed, with a small number of observations having very large view counts. Therefore, I applied a log transformation. The transformed distribution is substantially more symmetric and is more appropriate for use as an outcome in linear regression.

The distribution of `like_count`, `dislike_count`, and `comment_count` are also heavily right-skewed and contains zeros. I applied a square-root transformation to reduce the skewness while allowing zero values to remain in the data.

`favorite_count` is not appropriate as an outcome for linear regression because all non-missing observations have a value of zero. Therefore, the variable has no variation to model.

c. Fitting linear regression models

```
characteristics <- c(
  "funny",
  "show_product_quickly",
  "patriotic",
  "celebrity",
  "danger",
```

```

    "animals",
    "use_sex"
  )

  fit_model <- function(outcome, characteristic) {

    formula <- as.formula(
      paste(outcome, "~", characteristic, "+ year")
    )

    model <- lm(formula, data = youtube_clean)

    results <- summary(model)$coefficients

    c(
      estimate = results[2, "Estimate"],
      p_value = results[2, "Pr(>|t|)"]
    )
  }
}

```

```

view_results <- t(sapply(
  characteristics,
  function(x) fit_model("log_view_count", x)
))

view_results

```

	estimate	p_value
funny	0.53138094	0.20429571
show_product_quickly	0.20906409	0.60098197
patriotic	0.13435295	0.78811079
celebrity	-0.01756023	0.96571137
danger	0.66843693	0.09361301
animals	-0.17420727	0.64527849
use_sex	-0.29516669	0.49265404

Since all the p-values are larger than 0.05, none of the characteristics is significant. The commercial characteristic has potentially the largest positive estimated association with $\log(\text{view_counts})$ is danger.

```

like_results <- t(sapply(
  characteristics,

```

```

    function(x) fit_model("sqrt_like_count", x)
  ))

like_results

```

	estimate	p_value
funny	0.9143391	0.9176691
show_product_quickly	2.1684283	0.7970856
patriotic	9.7526762	0.3711089
celebrity	1.7772221	0.8377132
danger	7.3777897	0.3791831
animals	-1.7774914	0.8247240
use_sex	-10.3769462	0.2506394

Since all the p-values are larger than 0.05, none of the characteristics is significant.

```

dislike_results <- t(sapply(
  characteristics,
  function(x) fit_model("sqrt_dislike_count", x)
))

dislike_results

```

	estimate	p_value
funny	-0.6141524	0.8838144
show_product_quickly	1.7930184	0.6544879
patriotic	2.6620057	0.6076029
celebrity	-3.9659829	0.3355768
danger	4.1190731	0.3012801
animals	-0.1754750	0.9632990
use_sex	-4.1643559	0.3321021

Since all the p-values are larger than 0.05, none of the characteristics is significant.

```

comment_results <- t(sapply(
  characteristics,
  function(x) fit_model("sqrt_comment_count", x)
))

comment_results

```

	estimate	p_value
funny	-0.8834300	0.62608880
show_product_quickly	-0.6553496	0.70571103
patriotic	4.5341208	0.03599798
celebrity	0.6610099	0.70938541
danger	1.2534504	0.46804156
animals	-0.9427543	0.56672207
use_sex	-1.8394167	0.32039797

The p-value for `patriotic` is less than 0.05, thus it has a statistically significant association with `sqrt(comment_count)`. Since it is about 4.534, it has the largest and a positive association.

d. Estimating regression coefficients using matrix algebra

```
dat <- youtube_clean[
  complete.cases(youtube_clean[, c("log_view_count", "danger", "year"))],
]
```

```
X <- model.matrix(
  ~ danger + year,
  data = dat
)

y <- dat$log_view_count

beta_hat <- solve(t(X) %*% X) %*% t(X) %*% y

beta_hat
```

```
      [,1]
(Intercept) -31.28585897
dangerTRUE    0.66843693
year          0.02060294
```

```
model <- lm(
  log_view_count ~ danger + year,
  data = dat
)

coef(model)
```

(Intercept)	dangerTRUE	year
-31.28585897	0.66843693	0.02060294